

FRED TALKS

11th August 2026

CONTENTS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 1869 WORDS

The Night Watch

JAMES MICKENS · 2675 WORDS

The Reverse Information Paradox

SN SCRATCHPAD · 819 WORDS

2.1.227

CHANGELOG · 96 WORDS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 03 NOV 2025 · [SOURCE](#)

Is “LLM as a judge” overhyped? After years of implementing LLM judges for clients, I find more of them recognize the limitations.

LLM judges fail for the same reason any evaluation method fails: teams lack good data+evaluation to begin with. LLM judges aren't the shortcut teams hope. Instead, they are themselves a thing that needs data and monitoring to get use out of.

Let me walk through the pitfalls of LLM as a judge (for search) that you'll encounter.

LLM as a judge

When I talk about LLM as a judge, my focus is search. I mean creating a [judgment list](#). For search that means labeling how relevant a document is for a query. How relevant is the product “garden trowel” for a `q=shovels`? Maybe this “garden trowel” is less relevant than “iron spade”?

In the ancient past, to improve the search, I would be limited to a tiny hand-labeled judgment list. Labeling by hand is expensive. Datasets drift out of date quickly. New use cases arise. So we constantly need humans to relabel.

These costs inspire teams to use LLMs to build judgments, They:

1. Start with a ground truth of human labels
2. Build a prompt to approximate that ground truth
3. Apply those prompts to go beyond the ground truth to new query / documents

If done well, we can sit at our laptop, tinker with relevance algorithms, and continue to tune without bother humans for non-stop relabeling.

Sounds promising. But teams often ignore many of the assumptions and nuance underlying the task of LLM labeling. Let's go through them, and we'll see at the end, maybe LLMs should play a different role in evaluation?

Problem one - LLM evals don't know what's engaging to users

Users' ever-evolving lizard brains drive conversions. And while optimizing for just lizard brains would itself be a nightmare, so would only focusing on an LLM's opinion.

LLMs do a better job approximating *human evaluations*. In search, we call these evaluations *explicit* judgments. They use human labelers' basic knowledge of the world to label a result as *topically relevant* to the query or not.

Topical relevance is only one kind of relevance. As an example, for a search of "articles about harry potter". The article is either factually about Harry Potter or its not. We might also consider other gradations:

- Is the article well written?
- Is it article from an authoritative source?
- Is the article about a tangential topic? Dumbledore vs Gandalf etc?

All of which relate to topicality, authority, and knowledge. An LLM excels at this — LLMs live in a world of facts and knowledge.

But LLMs don't have limbic systems. So their evaluations come with limitations.

In real search applications, "relevance" goes beyond topicality. I worked at Reddit on search. A search for "harry potter" might mean anything from a memes, to drama about JK Rowling, to every kind of juicy controversy, spicy post, or oddity. People search for "cybertruck" not for reviews but to see silly videos.

Then there's the more mundane. I found at Shopify, users preferred plain or black clothing over flashy items based on their clicks. Unless you tell an LLM this (because you've done the analysis) an LLM won't intrinsically catch this.

Often the "topical" part of search is relatively easy. But engaging search, that captures these human oddities, must go beyond world-knowledge to at least get *some* of a user's lizard brain.

Problem two - the last 10% of disagreement is the important stuff

Many teams get excited when they immediately see 70-80% agreement with human labelers. With a bit of tuning, they can inch closer to 90%.

In my experience, that last bit of human-LLM disagreement matters. Those are the non-obvious cases. Search deals with a big hard negatives challenge. Those examples that deceptively seem relevant naive search algorithms (or labelers), but are, in fact irrelevant. That last 10-30% might be what more careful labelers would catch. Or represented in clickstream based labels.

For example on this furniture dataset, a LLM doesn't understand that a "bistro table" is meant for **outdoor** use in US furniture lingo. That's not obvious to an LLM: The LLM imagines I'm opening a restaurant.

Simple algorithms often capture that first 80-90%. It's the 10-20% you need great labels for. So pay attention not just to coverage. But for which use cases.

A 90% coverage might be good regression tests, it might not help you go beyond easier wins.

Problem three - sneaky overfitting

So far, I've mentioned a few strange cases the LLM judge might miss: a bistro table is meant for outdoor usage. Users preferring darker clothing items.

The savvy prompt engineer will want to fill the prompt with examples to help the LLM perform better on these cases.

But beware overfitting to these particular failure cases.

With every new rule, you push the model's meager attention away from the general problem, and towards your exceptions. Layer example after example, and you'll find yourself with a brittle system focused excessively on specific use cases.

Overfitting with prompts can be sneaky. Playing whack-a-mole with your eval data can degrade generality. You can get perfect results on the eval set, only to expand to unseen queries, and find the gains no longer hold.

To do a good job, you need holdout data you're not tuning against. A set of judgments you don't peak at. Because peaking might cause you to sneak a change into the prompt, making the holdout less useful as an independent validation step.

Add rules and exceptions carefully, you'll be surprised how easy it is to pull an LLM away from its generality.

Problem four - LLMs only evaluate the document it sees

LLMs only evaluate the document it sees. IE:

Query: outdoor seating

Product Name: Bistro Table

Product Description: Enjoy this lovely bistro table on your porch...

Is this enough information for evaluating the product? Is it what a user sees?

Suddenly, you decide to add new information about the product

Query: outdoor seating

Product Name: Bistro Table

Product Description: Enjoy this lovely bistro table on your porch...

Category: Outdoors

What are we evaluating truly?

- The products relevance?
- The usefulness of a set of product properties for determining relevance?
- How comprehensively you're describing products to an LLM?

Really it gets down to a classic problem in search. There's perceived relevance - the relevance of the product, as it appears in your product page. Then there's actual relevance - the detailed, thoughtful understanding of your product by a user. A lot depends not just on true relevance, but how it's presented to users.

Problem five - Your LLM judge can't be trusted on novel use cases

I assume the goal of LLM as a judge in to align to human labelers. But rarely do those judgments comprehensively represent every use case.

Search metrics with human evals don't go up-and-to-the-right. They seesaw as teams reactively label, solve, label, solve ad infinitum. In my experience it's rare to find a team build a true, comprehensive human labeled evals covering every use case. Even if we did, we'd need to account for evolving search use cases.

The flow usually goes like this: we want to solve a problem (search by product name). So we capture some judgments for that use case. Relevance starts low. We work a bit to improve this use case and it goes up. Yay!

But the boss comes to our desk and suggests we have a new problem to solve: search of furniture by the room (living room, etc). We gather more labels. We know we're bad at this use case. So search metrics go back down. Sad 😞

We solve that use case, it goes back up to where it was before. Repeat. Up and down. See-saw.

This presents a problem for an LLM judge. If our human labels only focus on a subset of use cases, **we're also aligning our LLM judges on a subset of use cases.**

Our LLM judge generalizes to just a piece of the problem. The LLM judge can't be trusted on a new use case until we measure it's abilities in that use case.

So if you seesaw its human labeling, you must seesaw the alignment of the LLM judge to the use cases we capture in human eval. As we adopt a new use case, we need more human labels. We need to retest the LLM judge. And so on.

The task of human labeling does not end with LLM Judges. In fact its even more crucial.

How to LLM as a judge

Here's how I use LLM judges:

- To extend labels on use cases I already measure decently to new queries (because I can trust alignment of the LLM on this use case)
- As a safeguard against regressions

I trust the LLM judge to flag a regression on the use cases my human labels cover. The LLM is aligned and measured against these. I don't trust the LLM to discover new, unexpected ways users search. I look to the users for that, not LLMs.

Still many teams don't even do decent monitoring of topical relevance. There are plenty of obviously broken search engines. The ones that would benefit from an LLM judge, likely would benefit from rounds of human labeling first needed before blindly trusting an LLM judge.

We already have automated data labeling at home kids

If we have enough labels to evaluate LLM as a judge on a use case, we have enough to train an ML model to recover those labels. We can then see a better place LLM's fit into evaluation:

- LLMs focus on factual, topical parts of the problem. But miss other parts of the problem
- LLMs might see a subset of the content features
- Other non-LLM features might help us better evaluate relevance (embedding similarity, text matches, etc)
- Other ground-truths (besides human) might be a better definition of relevance (ie engagement based labels)

Instead of LLM as a Judge, **why not Learning to Rank as a judge?** We seem to have forgotten that ranking models provide value beyond directly deploying them to production. They can help guide more manual search tuning as well.

I'm becoming more of a fan of LLM-for-feature-generation over LLM as a judge. LLMs can cover the topical part of relevance. You can ask them "Which of these product names is more relevant to the query?". The downstream, traditional ML model is agnostic to where the features come from. They could be an embedding similarity, LLM decision, lexical similarity, or more.

That's much more my interest these days. When someone says "LLM as a judge" I nod and say yes let's build that, but then go on and probably look into using LLM features heavily in a larger model. LLMs can be very useful for rapid feature development: they're promptable and easy to play with.

If you're interested in this approach, check out this [blog post](#) and the corresponding [Haystack talk](#). Also a talk from Shopify [choosing cross encoders for evaluation over LLMs](#)

Enjoy!

Enjoy softwareDoug in training course form!



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

The Night Watch

JAMES MICKENS · 11 APR 2026 · [SOURCE](#)

The Night Watch

JAMES MICKENS



James Mickens is a researcher in the Distributed Systems group at Microsoft's Redmond lab. His current research focuses on web applications,

with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters.

James received his PhD in computer science from the University of Michigan, and a bachelor's degree in computer science from Georgia Tech. mickens@microsoft.com

s a highly trained academic researcher, I spend a lot of time trying to advance the frontiers of human knowledge. However, as someone who was born in the South, I secretly believe that true progress is

A

a fantasy, and that I need to prepare for the end times, and for the chickens coming home to roost, and fast zombies, and slow zombies, and the polite zombies who say “sir” and “ma’am” but then try to eat your brain to acquire your skills. When the revolution comes, I need to be prepared; thus, in the quiet moments, when I’m not producing incredible scientific breakthroughs, I think about what I’ll do when the weather forecast inevitably becomes

RIVERS OF BLOOD ALL DAY EVERY DAY. The main thing that I ponder is who will be in my gang, because the likelihood of post-apocalyptic survival is directly related to the size and quality of your rag-tag group of associates. There are some obvious people who I’ll need to recruit: a locksmith (to open doors); a demolitions expert (for when the locksmith has run out of ideas); and a person who can procure, train, and then throw snakes at my enemies

(because, in a world without hope, snake throwing is a reasonable way to resolve disputes). All of these people will play a role in my ultimate success as a dystopian warlord philosopher. However, the most important person in my gang will be a systems programmer. A person who can debug a device driver or a distributed system is a person who can be trusted in a Hobbesian nightmare of breathtaking scope; a systems programmer has seen the terrors of the world and understood the intrinsic horror of existence. The systems programmer has written drivers for buggy devices whose firmware was implemented by a drunken child or a sober goldfish. The systems programmer has traced a network problem across eight machines, three time zones, and a brief diversion into Amish country, where the problem was transmitted in the front left hoof of a mule named Deliverance. The systems programmer has read the kernel source, to better understand the deep ways of the universe, and the systems programmer has seen the comment in the scheduler that says “DOES THIS WORK LOL,” and the systems programmer has wept instead of LOLed, and the systems programmer has submitted a kernel patch to restore

balance to The Force and fix the priority inversion that was causing MySQL to hang. A systems programmer will know what to do when society breaks down, because the systems programmer already lives in a world without law.

Listen: I'm not saying that other kinds of computer people

are useless. I believe (but cannot prove) that PHP developers have souls. I think it's great that database people keep trying to improve select-from-where, even though the only queries that cannot be expressed using select-from-where are inappropriate limericks from "The Canterbury Tales." In some way that I don't yet understand, I'm glad that theorists are investigating the equivalence between five-dimensional Turing machines and Edward Scissorhands. In most situations, GUI designers should not be forced to fight each other with tridents and nets as I yell "THERE ARE NO MODAL DATABASE LOGS IN SPARTA." I am like the Statue of Liberty: I accept everyone, even the wretched and the huddled and people who enjoy Haskell. But when things get tough, I need mission-critical people; I need a person who can wear night-vision goggles and descend from a helicopter on ropes and do classified things to protect my freedom while country music plays in the background. A systems person can do that. I can realistically give a kernel hacker a nickname like "Diamondback" or "Zeus Hammer." In contrast, no one has ever said, "These semi-

transparent icons are really semi-transparent! IS THIS THE WORK OF ZEUS HAMMER?"

I picked that last example at random. You must believe me when I say that I have the utmost respect for HCI people.

However, when HCI people debug their code, it's like an

art show or a meeting of the United Nations. There are tea breaks and witticisms exchanged in French; wearing a non-functional scarf is optional, but encouraged. When HCI code doesn't work, the problem can be resolved using grand theories that relate form and perception to your deeply personal feelings about ovals. There will be rich debates about the socioeconomic implications of Helvetica Light, and at some point, you will have to decide whether serifs are daring statements of modernity, or tools of hegemonic oppression that implicitly support feudalism and illiteracy. Is pinching-and-dragging less elegant than circling-and-lightly-caressing?

These urgent mysteries will not solve themselves. And yet, after a long day of debugging HCI code, there is always hope, and there is no true anger; even if you fear that your drop-down list should be a radio button, the drop-down list will suffice until tomorrow, when the sun will rise, glorious and

vibrant, and inspire you to combine scroll bars and left-clicking in poignant ways that you will commemorate in a sonnet when you return from your local farmer's market.

This is not the world of the systems hacker. When you debug a distributed system or an OS kernel, you do it Texas-style. You gather some mean, stoic people, people who have seen things die, and you get some primitive tools, like a compass and a rucksack and a stick that's pointed on one end, and you walk into the wilderness and you look for trouble, possibly while

using chewing tobacco. As a systems hacker, you must be prepared to do savage things, unspeakable things, to kill runaway threads with your bare hands, to write directly to network ports using telnet and an old copy of an RFC that you found in the Vatican. When you debug systems code, there are no high-level debates about font choices and the best kind of turquoise, because this is the Old Testament, an angry and monochromatic world, and it doesn't matter whether your Arial is Bold or Condensed when people are covered in boils and pestilence and Egyptian pharaoh oppression. HCI people discover bugs by receiving a concerned email from their therapist. Systems people discover bugs by waking up and discovering that their first-born children are missing and "ETIMEDOUT" has been written in blood on the wall.

What is despair? I have known it—hear my song. Despair is when you're debugging a kernel driver and you look at a memory dump and you see that a pointer has a value of 7. THERE IS NO HARDWARE ARCHITECTURE THAT IS ALIGNED ON

7. Furthermore, 7 IS TOO SMALL AND ONLY EVIL CODE WOULD TRY TO ACCESS SMALL NUMBER MEMORY.

Misaligned, small-number memory accesses have stolen decades from my life. The only things worse than misaligned, small-number memory accesses are accesses with aligned buffers, pointers, but impossibly large buffer lengths. Nothing ruins a Friday at 5 P.M. faster than taking one last pass through the log file and discovering a word-aligned buffer address, but a buffer length of NUMBER OF ELECTRONS IN THE UNI-

VERSE. This is a sorrow that lingers, because a 2893 byte read is the only thing that both Republicans and Democrats agree is wrong. It's like, maybe Medicare is a good idea, maybe not, but there's no way to justify reading everything that ever existed a jillion times into a mega-jillion sized array. This constant war on happiness is what non-systems people do not understand about the systems world. I mean, when a machine learning

algorithm mistakenly identifies a cat as an elephant, this is

actually hilarious. You can print a picture of a cat wearing an elephant costume and add an ironic caption that will entertain people who have middling intellects, and you can hand out copies of the photo at work and rejoice in the fact that everything is still fundamentally okay. There is nothing funny to print when you have a misaligned memory access, because your machine is dead and there are no printers in the spirit world.

An impossibly large buffer error is even worse, because these errors often linger in the background, quietly overwriting your state with evil; if a misaligned memory access is like a criminal burning down your house in a fail-stop manner, an impossibly large buffer error is like a criminal who breaks into your house, sprinkles sand atop random bedsheets and toothbrushes, and then waits for you to slowly discover that your world has been tainted by madness. Indeed, the common discovery mode for an impossibly large buffer error is that your program seems to

be working fine, and then it tries to display a string that should say "Hello world," but instead it prints "#a[5]:3!" or another syntactically correct Perl script, and you're like WHAT THE HOW THE, and then you realize that your prodigal memory accesses have been stomping around the heap like the Incredible Hulk when asked to write an essay entitled "Smashing Considered Harmful."

You might ask, "Why would someone write code in a grotesque language that exposes raw memory addresses? Why not use

a modern language with garbage collection and functional programming and free massages after lunch?" Here's the answer: Pointers are real. They're what the hardware understands. Somebody has to deal with them. You can't just place a LISP book on top of an x86 chip and hope that the hardware learns about lambda calculus by osmosis. Denying the existence of pointers is like living in ancient Greece and denying the existence of Krackens and then being confused about why none of your ships ever make it to Morocco, or Ur-Morocco,

or whatever Morocco was called back then. Pointers are like Krackens—real, living things that must be dealt with so that polite society can exist. Make no mistake, I don't want to write systems software in a language like C++. Similar to the Necro- nomicon, a C++ source code file is a wicked, obscure document that's filled with cryptic incantations and forbidden knowl- edge. When it's 3 A.M., and you've been debugging for 12 hours, and you encounter a virtual static friend protected volatile

templated function pointer, you want to go into hibernation and awake as a werewolf and then find the people who wrote the C++ standard and bring ruin to the things that they love. The C++ STL, with its dyslexia-inducing syntax blizzard of colons and angle brackets, guarantees that if you try to declare any reasonable data structure, your first seven attempts will result in compiler errors of Wagnerian fierceness:

```
Syntax error: unmatched thing in thing from std::nonstd::map<_Cyrillic, _$$$dollars>const
basic_string< epic_ mystery, mongoose_traits &lt; char>, _default_alloc_ <casual_ Fridays =
maybe>>
```

One time I tried to create a list<map<int>>>, and my syntax errors caused the dead to walk among the living. Such things are clearly unfortunate. Thus, I fully support high-level languages in which pointers are hidden and types are strong and the declaration of data structures does not require you to solve a syntactical puzzle generated by a malevolent extraterrestrial species. That being said, if you find yourself drinking a martini and writing programs in garbage-collected, object-oriented Esperanto, be aware that the only reason that the Esperanto runtime works is because there are systems people who have exchanged any hope of losing their virginity for the exciting opportunity to think about hex numbers and their relationships

with the operating system, the hardware, and ancient blood rituals that Bjarne Stroustrup performed at Stonehenge.

Perhaps the worst thing about being a systems person is that other, non-systems people think that they understand the daily tragedies that compose your life. For example, a few weeks ago, I was debugging a new network file system that my research group created. The bug was inside a kernel-mode component, so my machines were crashing in spectacular and vindic-

tive ways. After a few days of manually rebooting servers, I had transformed into a shambling, broken man, kind of like a computer scientist version of Saddam Hussein when he was pulled from his bunker, all scraggly beard and dead eyes and florid, nonsensical ramblings about semi-imagined enemies.

As I paced the hallways, muttering Nixonian rants about my code, one of my colleagues from the HCI group asked me what my problem was. I described the bug, which involved concurrent threads and corrupted state and asynchronous message delivery across multiple machines, and my coworker said,

“Yeah, that sounds bad. Have you checked the log files for errors?” I said, “Indeed, I would do that if I hadn’t broken every component that a logging system needs to log data. I have a network file system, and I have broken the network, and I have broken the file system, and my machines crash when I make eye contact with them. I HAVE NO TOOLS BECAUSE I’VE DESTROYED MY TOOLS WITH MY TOOLS. My only logging

option is to hire monks to transcribe the subjective experience of watching my machines die as I weep tears of blood.” My co-worker, in an earnest attempt to sympathize, recounted one of his personal debugging stories, a story that essentially involved an addition operation that had been mistakenly replaced with

a multiplication operation. I listened to this story, and I said, “Look, I get it. Multiplication is not addition. This has been known for years. However, multiplication and addition are at least related. Multiplication is like addition, but with more addition. Multiplication is a grown-up pterodactyl, and addition is a baby pterodactyl. Thus, in your debugging story, your code is wayward, but it basically has the right idea. In contrast, there is no family-friendly GRE analogy that relates what my code should do, and what it is actually doing. I had the mod-

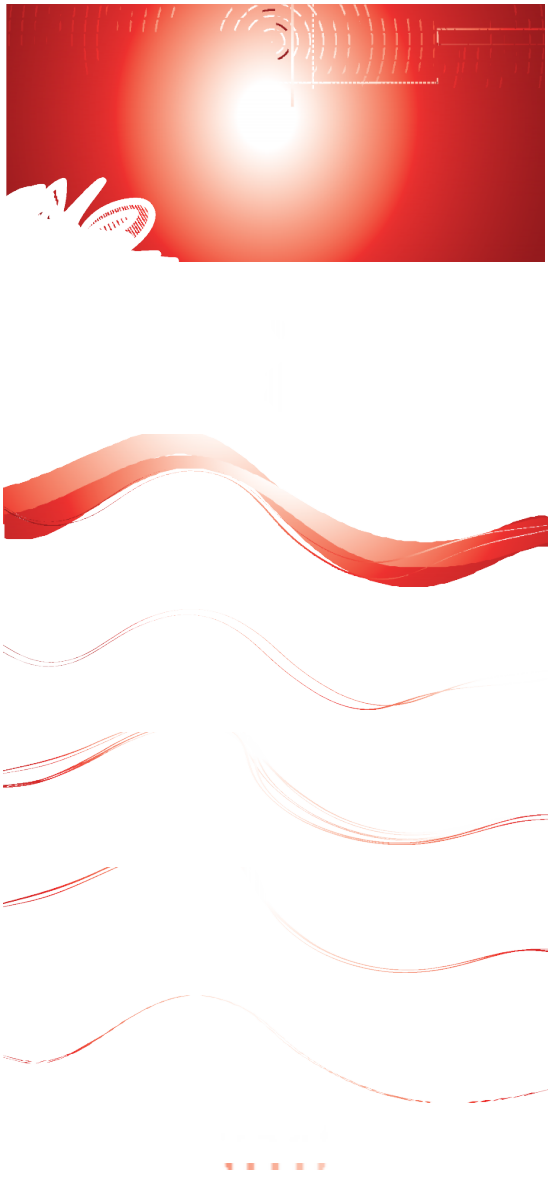
est goal of translating a file read into a network operation, and now my machines have tuberculosis and orifice containment issues. Do you see the difference between our lives? When you asked a girl to the prom, you discovered that her father was a cop. When I asked a girl to the prom, I DISCOVERED THAT HER FATHER WAS STALIN.”

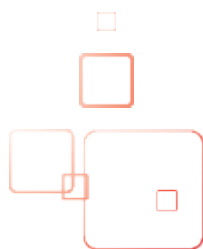
In conclusion, I’m not saying that everyone should be a systems hacker. GUIs are useful. Spell-checkers are useful. I’m glad that people are working on new kinds of bouncing icons because they believe that humanity has solved cancer and homelessness and now lives in a consequence-free world

of immersive sprites. That’s exciting, and I wish that I could join those people in the 27th century. But I live here, and I live now, and in my neighborhood, people are dying in the streets. It’s like, French is a great idea, but nobody is going to invent French if they’re constantly being attacked by bears. Do you see? SYSTEMS HACKERS SOLVE THE BEAR MENACE.

Only through the constant vigilance of my people do you get

the freedom to think about croissants and subtle puns involv- ing the true father of Louis XIV.
So, if you see me wandering the halls, trying to explain synchronization bugs to confused monks, rest assured that every day, in every way, it gets a little better. For you, not me. I'll always be furious at the number 7, but such is the hero's journey.





u
s
e
n
x

THE ADVANCED
COMPUTING SYSTEMS
ASSOCIATION



Do you know about the USENIX Open Access Policy?

USENIX is the first computing association to offer free and open access to all of our conferences proceedings and videos. We stand by our mission to foster excellence and innovation while supporting research with a practical bias. Your membership fees play a major role in making this endeavor successful.

Please help us support open access.

Renew your USENIX membership

and ask your colleagues to join or renew today!

www.usenix.org/membership

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · [SOURCE](#)

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow [famously described](#) a paradox in the market for information. “Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost.” In Arrow’s “Information Paradox,” the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow’s paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from “exhaust,” the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It’s the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek’s sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it’s imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: “What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it’s not being transferred to someone else.” The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization’s data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm’s right to align models to their enterprise accountability obligations.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what “good” looks like inside the organization. Also, retain ownership of your organization’s memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company's knowledge.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company "veteran" capability remain with you even if a given "generalist" model is taken away?

Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.

2.1.227

CHANGELOG · 11 AUG 2026 · [SOURCE](#)

- Fixed feature flags being evaluated without the user's subscription tier when a session started with an expired login token, which could wrongly prompt Max plan users to enable usage credits for Fable
- Fixed every Bash command failing under `claude-code-action` with `allowed_non_write_users` on GitHub-hosted runners
- Fixed `/tui` bringing back a conversation that had been rewound to before its first message
- Improved slash-command menu: blue now marks only the selected row, matched characters are bolded instead of recolored, and emoji or accented names keep their glyphs
- Improved performance: fewer event-loop stalls on file-not-found suggestions and at-mention size checks